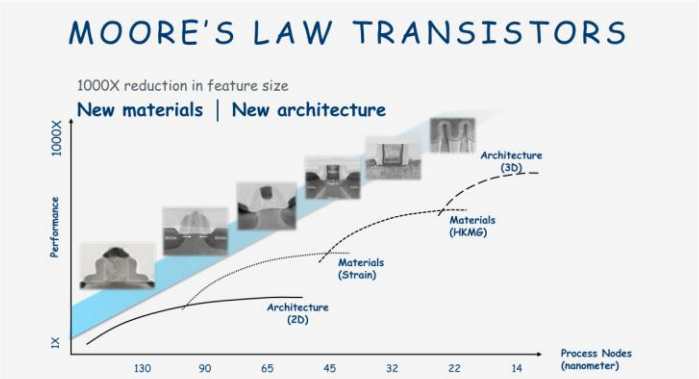
面向 LLM 部署的算法级协同优化：混合精度、层级共享与轻量化稀疏注意力

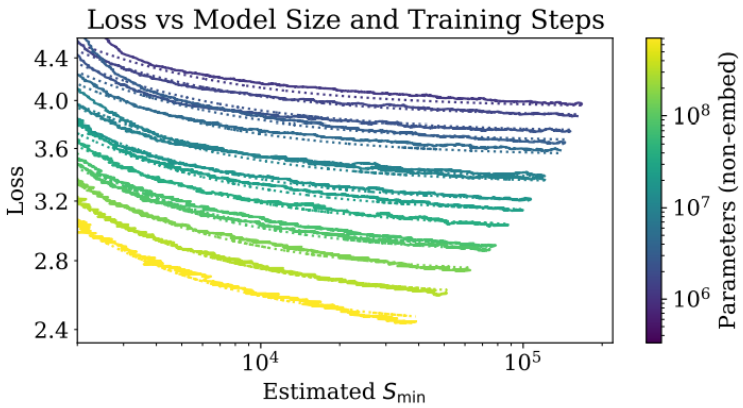
黄瀚韬

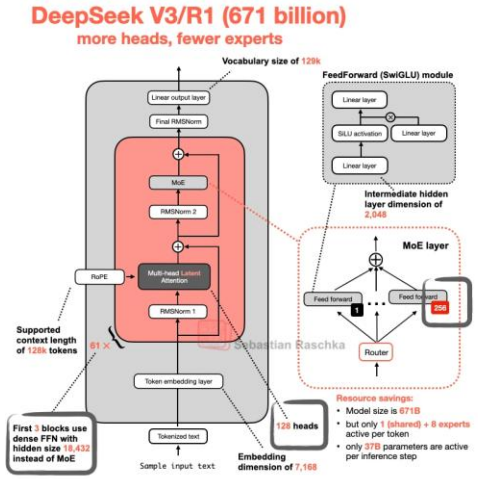
字节跳动
异构计算工程师

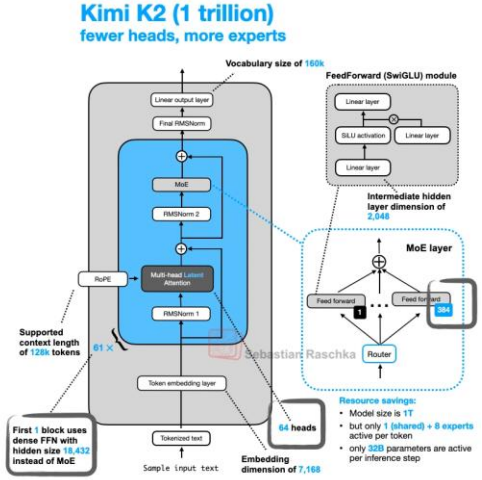
LLM模型趋势（一）
— Scaling still works

- 芯片的Scaling Law: 摩尔定律影响着芯片的迭代
- LLM Scaling Law LLM：参数越多，模型越大，loss 越小， 精度越好
- 稀疏模型MOE 逐渐成为主流方案







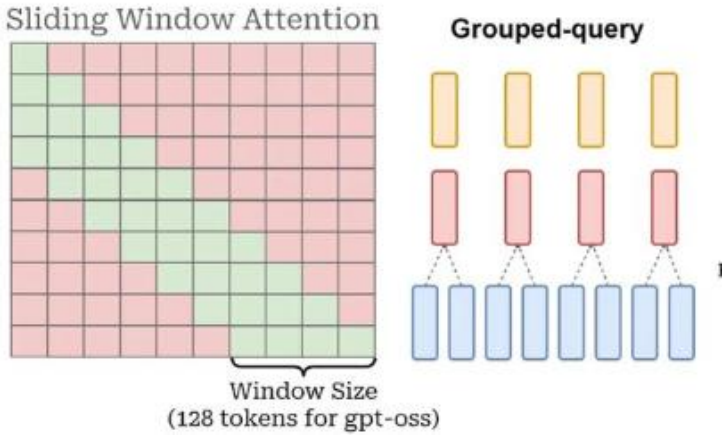


https://singularitykchen.github.io/blog/2020/07/12/Workshop-Moore_s-Law-in-the-age-of-AI-Chips/

LLM模型趋势（二）

— Sparse /Linear Attention is a must.

高效Attention、稀疏Attention模型



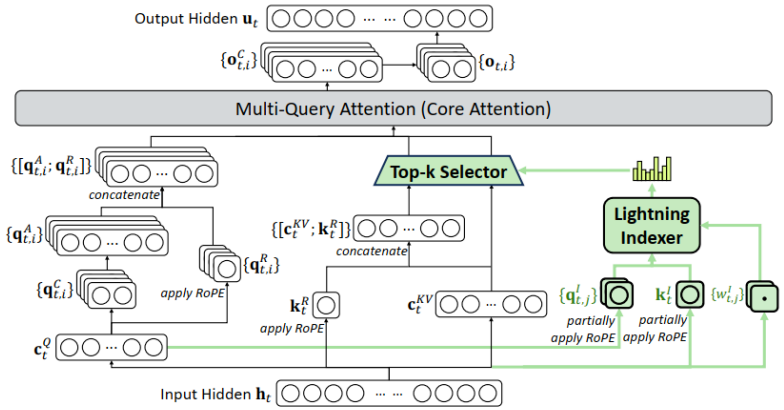
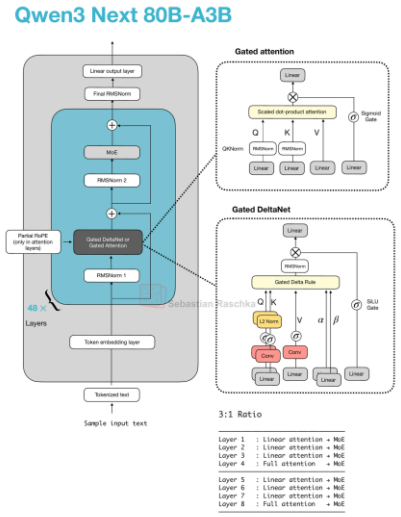
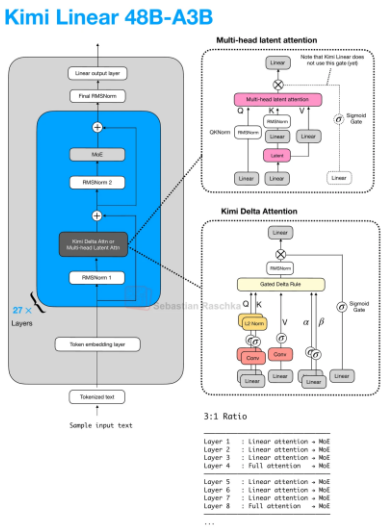


Figure 1 | Attention architecture of DeepSeek-V3.2-Exp, where DSA is instantiated under MLA. The green part illustrates how DSA selects the top-k key-value entries according to the indexer.





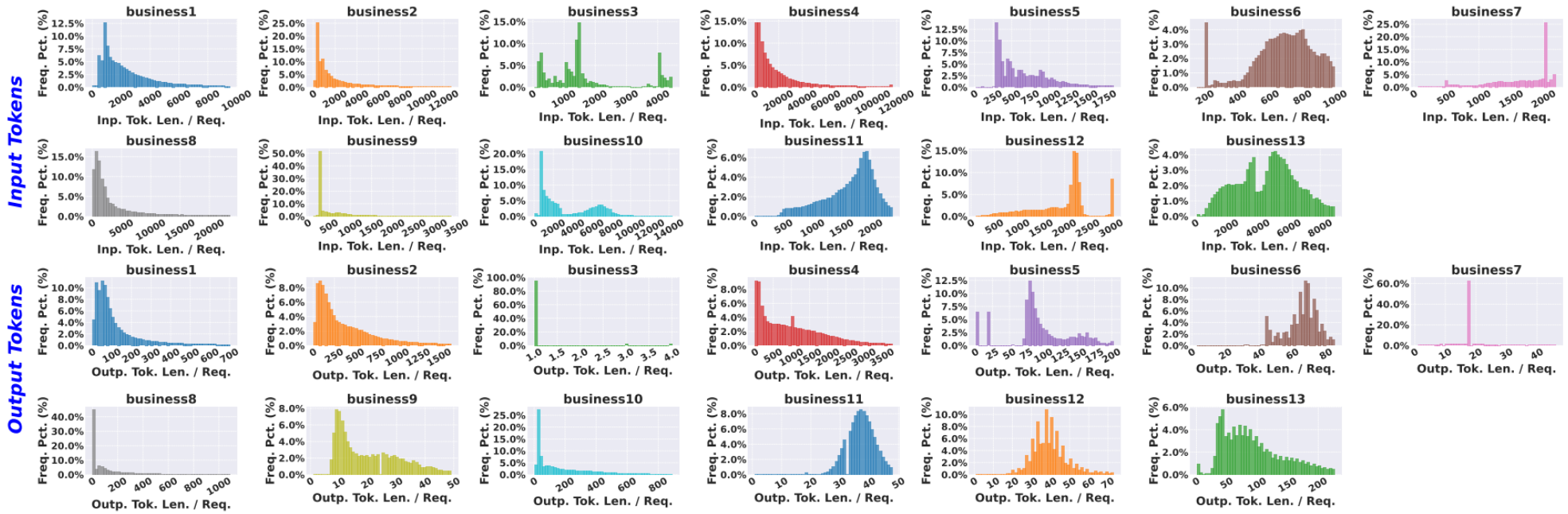
OpenAI GPT-OSS Recent Window

DeepSeek DSA

Linear Attention

互联网业务趋势
—围绕大模型构筑端到端的生态，业务多样，逻辑复杂

- 坚持基础模型创新，围绕大模型构筑端到端的生态，业务多样
- 没有一个单一Shape适用于所有业务；每个请求的输入长度与业务强相关；
- 长远的看，序列的长度越来越长，考虑的上下文的token越来越多。



J. Cai *et al.*, “Characterizing Cloud-Native LLM Inference at Bytedance and Exposing Optimization Challenges and Opportunities for Future AI Accelerators,” *IEEE* ***HPCA***, Sydney, Australia, 2026.

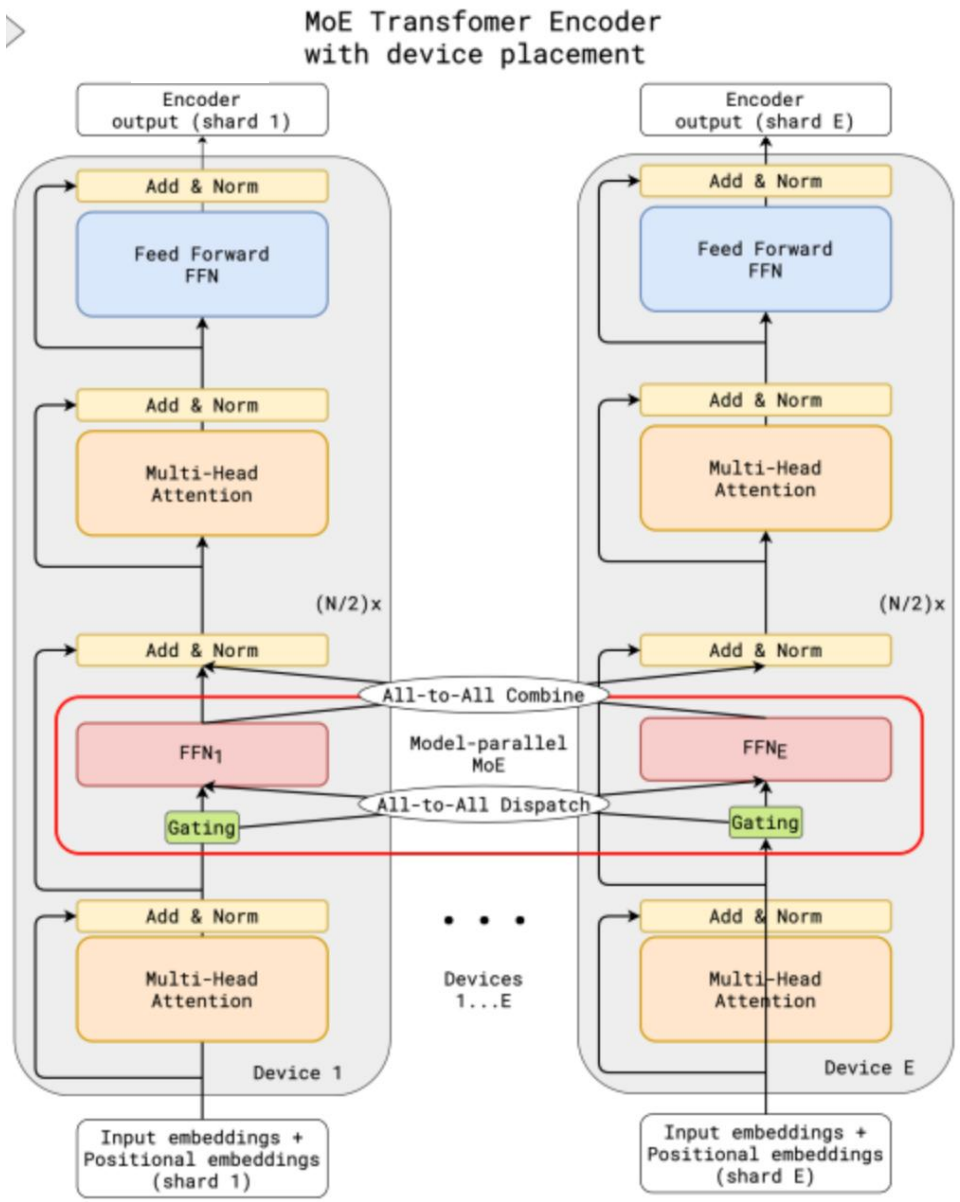
LLM 部署的算法级协同优化

LLM 预训练阶段	LLM FT 训练阶段	LLM 部署阶段	**LLM 的不同阶段**
量化	投机采样	稀疏	低秩	**LLM 部署优化技术**
FP8 训练、TurboQuant	DSA 稀疏训练	Eagle 1,2,3; MTP	Hshare/SALS	**相关的代表作**

量化技术

量化是最直接而且收益最容易拿到的算法优化技术。

1. **Attention 量化**
   - Sage Attention, 低精softmax, KV 量化
2. **MOE 量化**
   - Smooth quant, w4a4 (NVFP4, INT4)
3. **通信量化**
   - Dispatch 4 bit/ Combine 8 bit

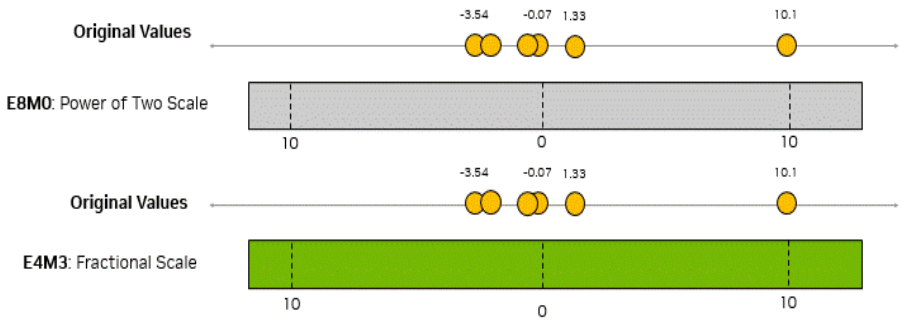


量化数据类型

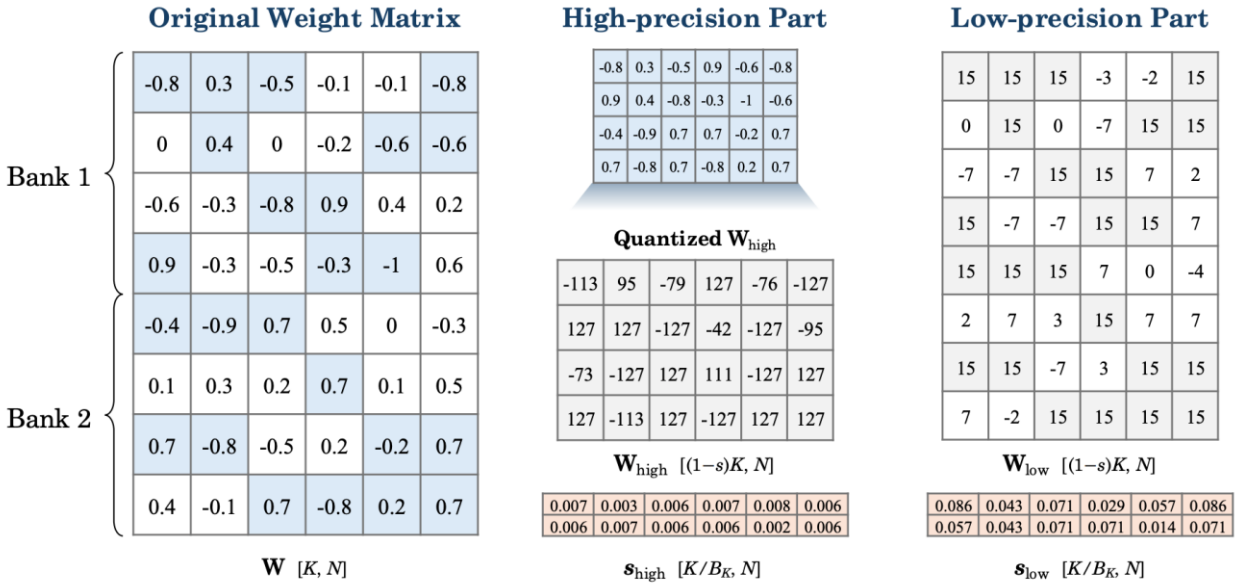
OCPMX 数据类型

Format Name	Element Data Type	Element Bits (d)	Scaling Block Size (k)	Scale Data Type	Scale Bits (w)
MXFP8	FP8 (E5M2)	8	32	E8M0	8
	FP8 (E4M3)				
MXFP6	FP6 (E3M2)	6	32	E8M0	8
	FP6 (E2M3)				
MXFP4	FP4 (E2M1)	4	32	E8M0	8
MXINT8	INT8	8	32	E8M0	8

NVIDIA NV FP4 数据类型



SQ format 高精度和低精度的混合数据类型



[1] NVFP4: https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/
[2] Sq-format: https://arxiv.org/pdf/2512.05409
[3] MX-Fformat: https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf

量化敏感度分析

参考Optimal Brain Quantization(OBQ):

泰勒展开式 the error function(or loss function):

$$\delta E = \left(\frac{\partial E}{\partial \mathbf{w}}\right)^T \cdot \delta \mathbf{w} + \boxed{\frac{1}{2}\delta \mathbf{w}^T \cdot \mathbf{H} \cdot \delta \mathbf{w}} + \boxed{O(\| \delta \mathbf{w} \|^3)}$$

Ignored

GPTQ: 层间量化精度最优.

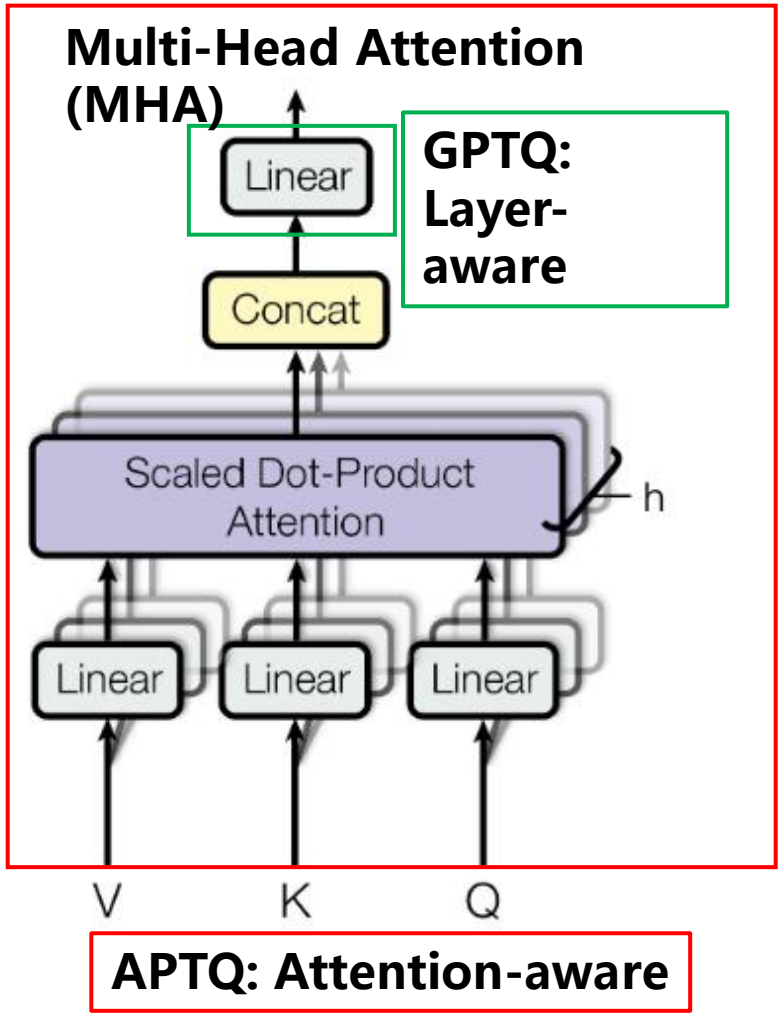
$$E = \| \mathbf{WX} - \widehat{\mathbf{W}}\mathbf{X} \|_2^2$$

Local Optima *局部最优值*

APTQ: Block间量化精度最优

$$E = \| \mathbf{MHA}(\mathbf{W}, \mathbf{X}) - \mathbf{MHA}(\widehat{\mathbf{W}}, \mathbf{X}) \|_2^2$$

Global Optima *全局最优值*



APTQ量化原理

权重只根据输入进行更新

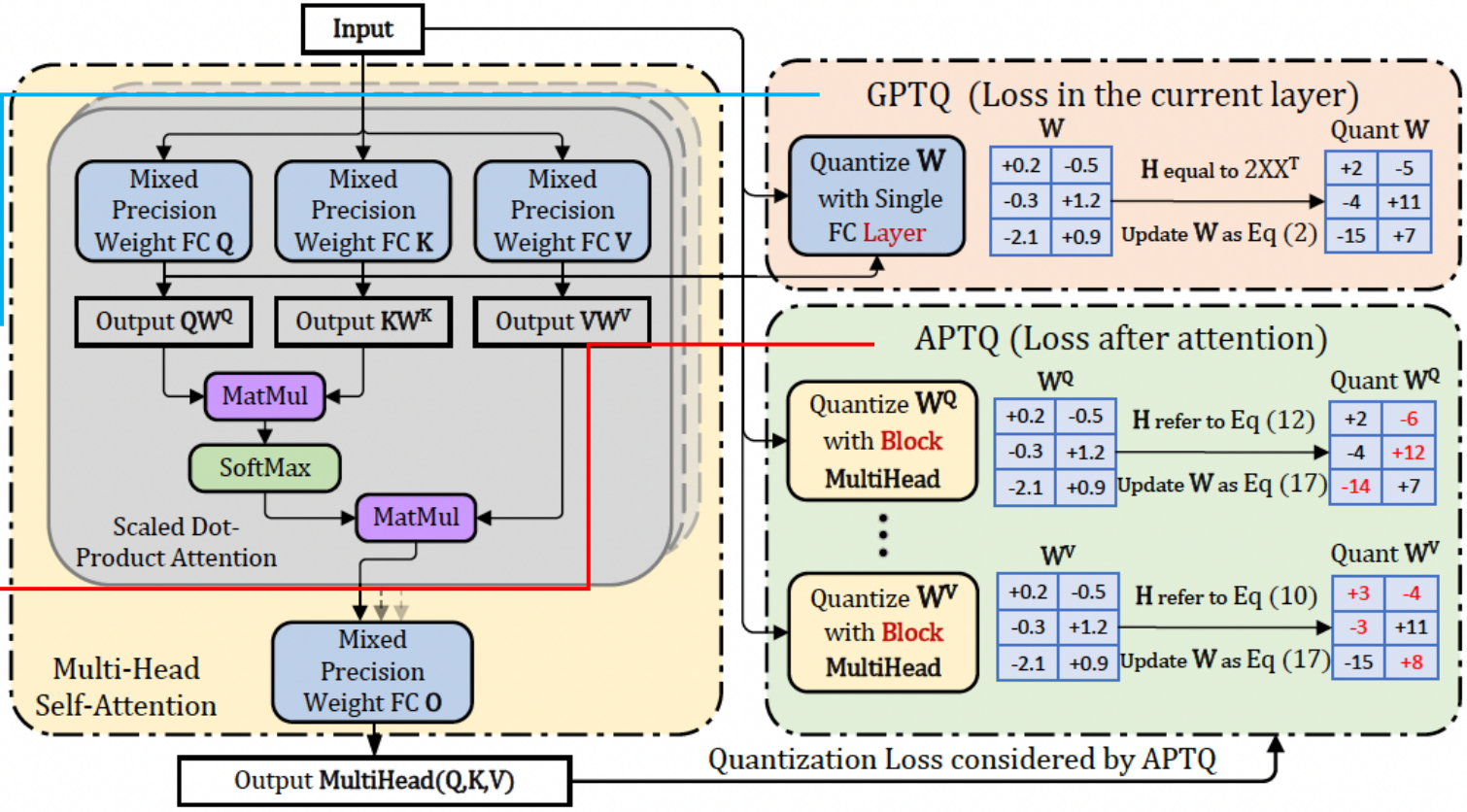
GPTQ 权重更新公式：

$$H_F = 2X_F X_F^T$$

APTQ 权重更新公司：

$$H_{\hat{w}} = 2 \cdot [F'(\hat{w}) \cdot F'(\hat{w})^T]$$

权重根据整个注意力层的梯度进行更新



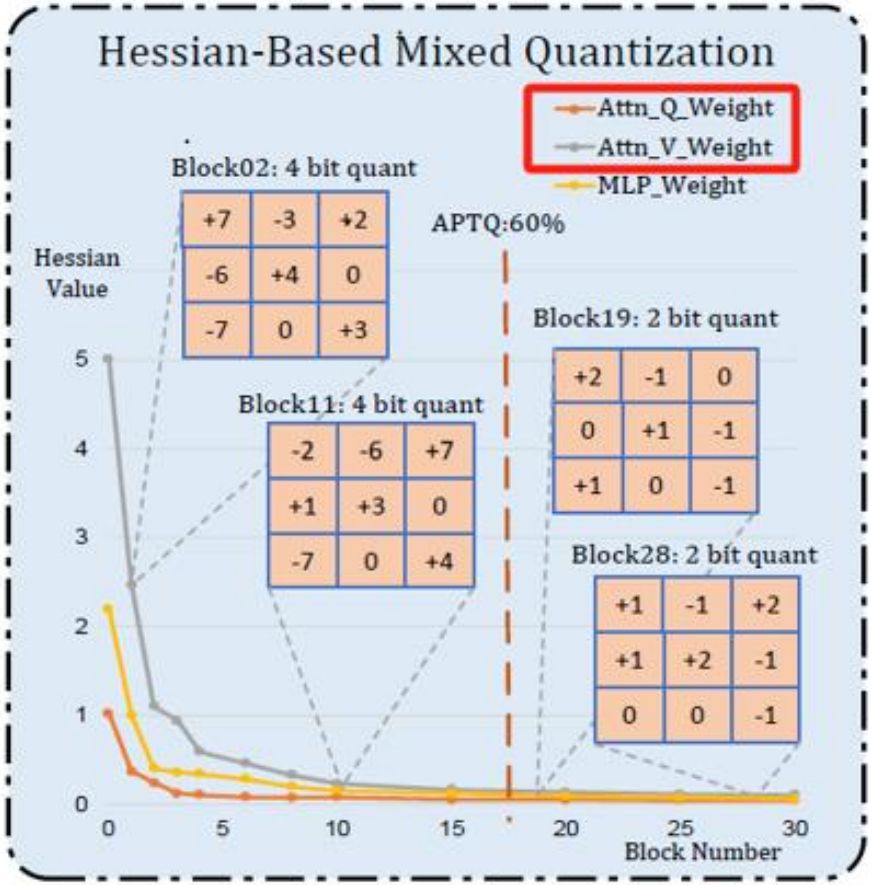
量化模型效果

混合精度量化度量标准

混合精度的指标：平均 ***Hessian Trace 值***

$$Average\ bits = 4 \times R + 2 \times (1 - R)$$

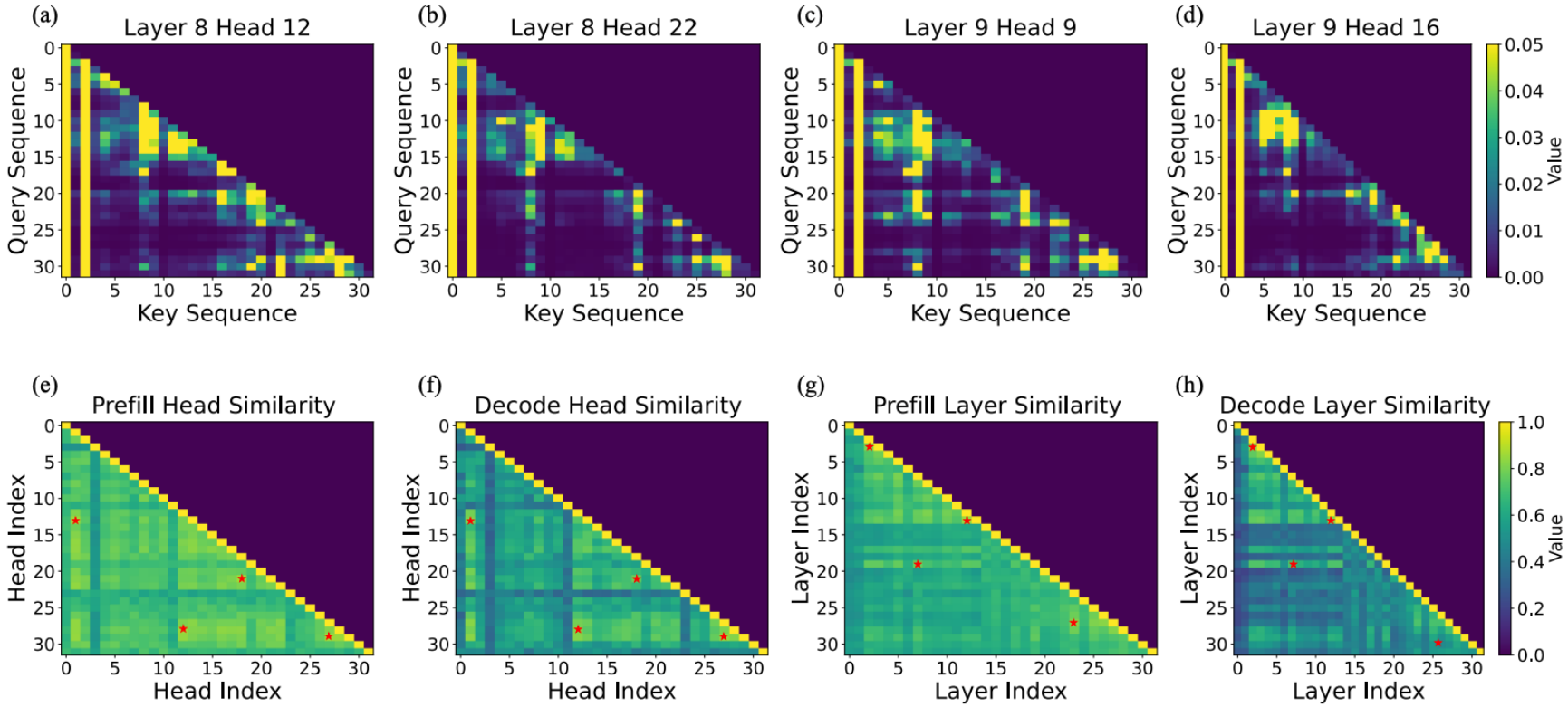
这里 R 代表量化成4bit 的比例， 其余量化成2比特 (右图, $R = 0.6$)。



稀疏Attention

$$y = \text{softmax}\left(\frac{Q \cdot K_{CT}^{T}}{\sqrt{d_h}}\right) \cdot V_{CT}$$

- Softmax 归一化，在长序列会引入低关注的token. 但是挑选critical token 本身会引入IO的瓶颈。
- 不同layer和head 之间，都有相似的关键token的挑选。
- Prefill和Decode阶段都有相似的关键 token挑选pattern.



稀疏共享策略

Model	Architecture	Method	GSM8K(flexible/strict)↑	COQA ↑	Complexity* ↓
LLaMA2-7b-chat	MHA	Original	0.2297/0.2297	0.5997	-
		StreamingLLM	0.0591/0.0023	0.3577	0
		H2O	0.0986/0.0144	0.4952	$\mathcal{O}(1)$
		Quest	0.0478/0.0462	0.5713	$0.125T$
		DS	0.1713/0.1706	0.5978	$0.0625T$
		Ours (7/8-3/4-1/2)	**0.1835/0.1721**	**0.5982**	$0.021T$
		Ours (3/4-3/4-1/2)	0.1803/0.1703	0.5898	$0.018T$
		Ours (1/2-1/2-1/2)	0.1524/0.1145	0.5672	$0.008T$

- **跨层共享 (Across Layers)**：不同层级之间共享相同的关键令牌索引, 其中 n_l 为总层数，k_l 为与其他层共享关键 KV 索引的层数, ratio = $1-k_l/n_l$
- **跨头共享 (Across Heads)**：同一层内的不同注意力头（如头 h_1 和 h_2）之间共享关键令牌索引
- **跨查询共享 (Across Queries)**：相邻的查询步（如步 q_1 和 q_2）之间复用之前计算出的关键令牌索引

低秩： ROPE对计算的影响

低秩投影（Low-rank projection）与 RoPE（旋转位置嵌入）前后的交互方式不同

- RoPE 后的投影（Post-RoPE projection）可以融合进旋转后的查询（Query）中，并直接在潜空间（Latent Space）中使用
- RoPE 前的投影（Pre-RoPE projection）则需要先重构全维度键（Keys），然后才能应用 RoPE

$$\mathbf{Q}_i\,\mathbf{K}_j^\top \approx \underbrace{\mathbf{Q}\,\mathbf{R}_i\left(\mathbf{R}_j^\top\,\mathbf{U}\mathbf{U}^\top\right)\mathbf{K}^\top}_{\text{pre-RoPE}} \quad \text{or} \quad \underbrace{\mathbf{Q}\,\mathbf{R}_i\left(\mathbf{U}\mathbf{U}^\top\,\mathbf{R}_j^\top\right)\mathbf{K}^\top}_{\text{post-RoPE}},$$

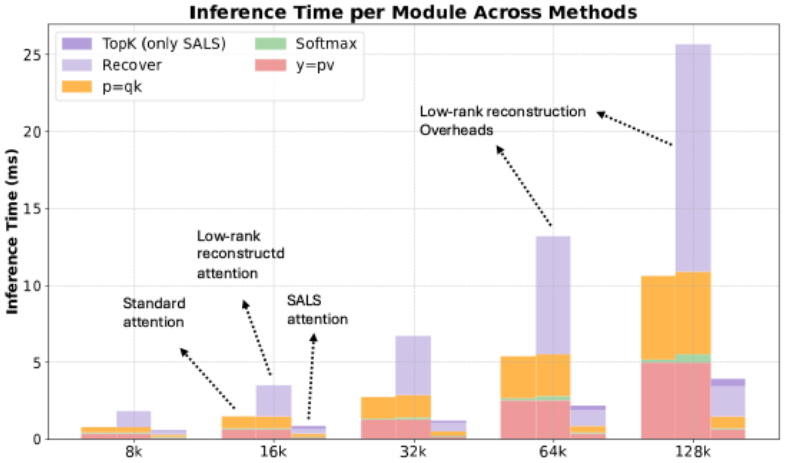
低秩：ROPE对低秩的影响

挑战1: 在 RoPE（旋转位置编码）之后进行压缩会导致高方差增加

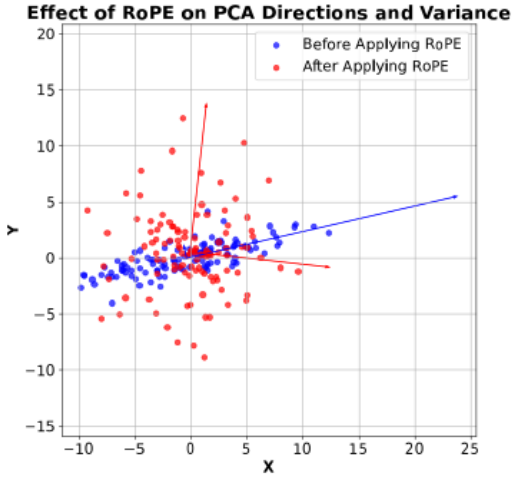
- RoPE 会对 Key 向量进行旋转和分散，这增加了向量的有效秩（effective rank），并破坏了其原有的低秩结构

挑战2: 在 RoPE 之前进行重构会产生计算开销

- 低秩形式的 KV 缓存必须在应用 RoPE 之前完成完全重构，这会带来额外的计算成本

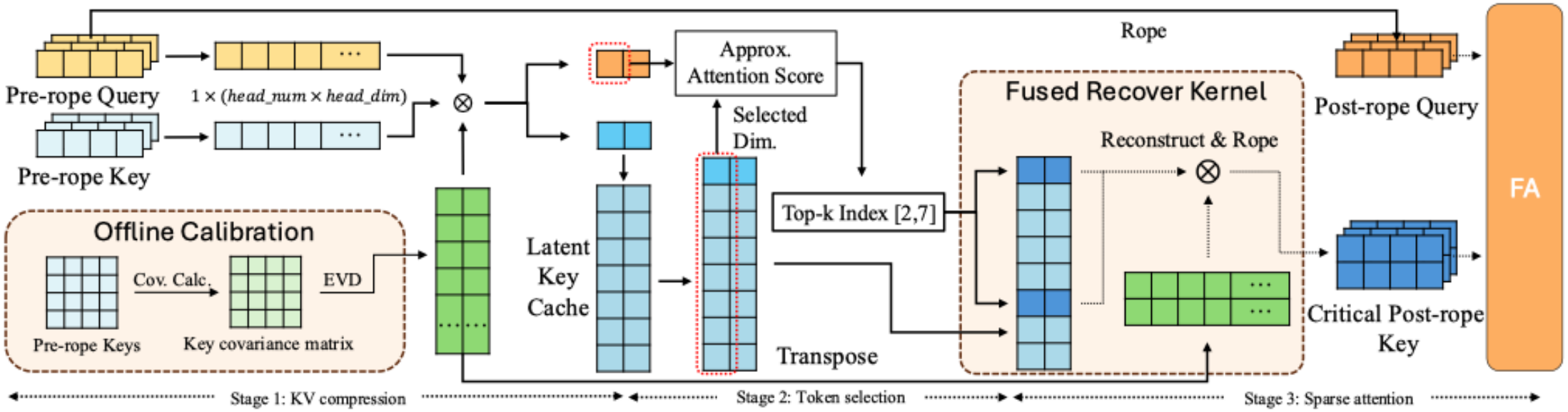


(a)



(b)

低秩+稀疏：SALS 的稀疏化方案



- **潜在 KV 压缩 (Latent KV Compression)**：将所有多头 Key（键）和 Query（查询）投影到一个共享的低秩潜在空间中，从而显著减小 KV 缓存（KV-cache）的占用体积
- **潜在令牌选择 (Latent Token Selection)**：直接在不含 RoPE（旋转位置编码）的潜在空间中计算注意力分数，并筛选出前 K 个最重要的令牌（Tokens）
- **稀疏注意力重构 (Sparse Attention Reconstruction)**：仅将选定的令牌重构回完整维度，随后应用 RoPE，并在这个精简的子集上高效地完成最终的注意力计算

低秩+稀疏：SALS的压缩效果

Table 2 Evaluation on GSM8K and CoQA datasets for LLaMA2–7B–chat

Method	GSM8K (strict/flexible)↑	CoQA↑	Memory Access↓	Comp. ratio↓
baseline	0.2335 / 0.2335	0.5997	1.00	1.00
KIVI-4	0.2297 / 0.2305	0.5992	0.31	0.31
KIVI-2	0.2047 / 0.2047	0.6010	0.19	0.19
Palu-30%(3bit)	0.1713/0.1759	0.5938	0.14	0.14
Palu-50%(3bit)	0.0614/0.0879	0.5560	0.09	**0.09**
SALS-25%	**0.2312 / 0.2343**	0.5975	0.13	0.28
SALS-12.5%	0.2176 / 0.2229	**0.6070**	**0.07**	0.15

在GSM8K和COQA的数据集下KV cache的压缩程度。

- 表 2 展示了 LLaMA2-7B-chat 模型在基准测试（GSM8K、CoQA）上的评估结果。

我们可以看到，尽管 Palu 实现了最高的压缩率，但其准确率下降不可忽视，特别是在 GSM8K 数据集上。

- SALS 提供了两种设置下的结果：25% KV 缓存压缩率（SALS-25%）和 12.5% 压缩率（SALS-12.5%）。

与 KIVI 和 Palu 相比，SALS-25% 取得了最佳的准确率，且相较于基准（baseline）的性能损失可忽略不计。

低秩+稀疏：SALS的稀疏倍率

不同的稀疏（上表）、KV压缩（下表）算法在Llama2-7B的精度效果

Method	Single-QA	Multi-QA	Summari-zation	Few-shot	Synthetic	Code	Avg	Memory Access↓
baseline	24.50	22.18	23.64	62.80	6.50	56.29	32.65	1.00
Double Sparse	24.78	**22.72**	**24.70**	61.84	4.17	51.66	31.64	0.16
HShare	24.59	22.18	24.54	61.69	4.74	53.26	31.83	0.14
Loki	24.57	18.09	24.37	**63.43**	4.75	56.49	31.95	0.19
SALS-25%	24.37	22.50	23.40	63.01	**6.50**	**53.80**	**32.26**	0.11
SALS-12.5%	**24.81**	22.52	23.08	62.84	5.00	53.58	31.97	**0.06**

Model	Method	Single-QA	Multi-QA	Summari-zation	Few-shot	Synthetic	Code	Avg	Memory Access↓
LLaMA2-7B-chat	baseline	24.50	22.18	23.64	62.80	6.50	56.29	32.65	1.00
	KIVI-4bit	**24.76**	22.22	**23.40**	62.67	6.75	**56.42**	**32.70**	0.31
	Palu-30% (3bit)	22.36	22.01	22.66	60.39	**7.50**	50.53	30.91	0.13
	SALS-25%	24.37	**22.50**	**23.40**	**63.01**	6.50	53.80	32.26	**0.11**
	KIVI-2bit	23.11	21.40	22.67	62.81	**5.75**	**55.84**	31.93	0.19
	Palu-50% (3bit)	20.48	22.18	21.44	54.62	**5.75**	37.07	26.92	0.09
	SALS-12.5%	**24.81**	**22.52**	**23.08**	**62.84**	5.00	53.58	**31.97**	**0.06**
Mistral-7B-v0.2	baseline	36.43	29.65	28.06	66.72	44.87	52.96	43.12	1.00
	KIVI-4bit	36.34	29.85	**28.09**	**66.81**	**44.67**	**52.90**	**43.11**	0.31
	Palu-30% (3bit)	29.48	36.40	27.20	65.73	53.19	40.77	34.74	0.13
	SALS-25%	**36.61**	**29.92**	27.57	66.63	43.54	52.49	42.79	**0.11**
	KIVI-2bit	**35.34**	28.63	**27.72**	66.77	**39.68**	**52.63**	**41.80**	0.19
	Palu-50% (3bit)	26.73	**32.72**	25.73	63.25	18.57	44.43	35.71	0.09
	SALS-12.5%	35.18	29.88	27.18	**66.98**	35.32	51.39	40.99	**0.06**

SALS 在多种任务（QA、总结、Few-shot、代码等）中表现出了**极强的鲁棒性**：

- **接近基准：** 在 LLaMA2-7B-chat 上，SALS-25% 的平均分（32.26）与 baseline（32.65）非常接近，性能损失极其微小。
- **压缩效率：** SALS-12.5% 仅需 **0.06x** 的内存访问量，相比之下，基准模型（baseline）为 1.00x。这意味着它在显存带宽占用上实现了约 **16.6** 倍 的优化。
- **横向对比：** 在相同的内存访问水平下，SALS 的表现优于其他压缩方案。例如，在 LLaMA2-7B 模型中，SALS-12.5% (0.06) 的平均分 (31.97) 远高于同类极低内存占用的 Palu-50% (26.92)

GPU1 的加速效果

Attention 算子在不同算法和输入配置下的延时

Config (ms)	Flash-attn [7]	Loki [20]	Double-sparse [29]	Hshare [25]	SALS-25%	SALS-12.5%
bs=8, 1k	0.230	0.248	0.138	0.134	0.409 ± 0.093	0.357 ± 0.051
bs=8, 2k	0.830	0.464	0.237	0.430	0.430 ± 0.018	0.359 ± 0.012
bs=8, 4k	1.630	1.102	0.724	0.576	0.530 ± 0.008	0.439 ± 0.016
bs=16, 1k	0.440	0.452	0.223	0.134	0.415 ± 0.008	0.360 ± 0.011
bs=16, 2k	1.630	0.864	0.434	0.246	0.552 ± 0.040	0.444 ± 0.019
bs=16, 4k	3.230	2.101	1.319	1.067	0.757 ± 0.026	0.565 ± 0.013

- **随序列长度增加，加速效果愈发显著,** SALS 在长序列任务中表现出了极佳的计算缩放性（Scalability）：
对比 Flash-attention： 在 bs=16, 4k 的设置下，Flash-attention 的延迟为 **3.230 ms**，而 SALS-12.5% 仅为 **0.565 ms**。这意味着 SALS 实现了约 **5.7 倍** 的速度提升。

- **内存受限场景下的优势**
高 Batch Size 表现： 在 bs=16 时，SALS 的加速优势比 bs=8 时更为明显。这证明了 SALS 通过显著减少 KV Cache 的内存读取量，成功缓解了推理过程中的 **Memory-bound（内存带宽受限）** 问题。

核心观点总结

- **需求演进与瓶颈：**

随着大语言模型应用场景的深化，超长序列处理已成常态。这导致 KV Cache 规模激增，对**存储容量、访存带宽**以及逐 Token 生成延迟（TPOT）构成了严峻的性能挑战。

- **单一优化的局限：**

传统的单一算法维度（如单纯的量化、稀疏化或低秩分解）在处理超大规模模型时，往往面临“精度损耗”与“压缩倍率”的权衡瓶颈，难以实现极致的推理性能。

- **协同优化的突破：**

本报告提出一种“量化混合精度、冗余挖掘、隐空间重构”**三位一体的深度协同方案。通过算法层面的多维整合，在保持模型精度的前提下，实现了**权重存储、稀疏调度开销及 KV 缓存占用的系统级优化。

Thank you

黃瀚韜